2026

TdP

Graphical Interfaces

Intro to Flet library, MVC Pattern

Giuseppe Averta
Carlo Masone
Francesca Pistilli
Davide Buoso
Gaetano Falco


```python
import flet
from flet import IconButton, Page, Row, TextField, icons

def main(page: Page):
    page.title = "Flet counter example"
    page.vertical_alignment = "center"

    txt_number = TextField(value="0", text_align="right",
width=100)

    def minus_click(e):
        txt_number.value = int(txt_number.value) - 1
        page.update()

    def plus_click(e):
        txt_number.value = int(txt_number.value) + 1
        page.update()

    page.add(
        Row(
            [
                IconButton(icons.REMOVE, on_click=minus_cli
                txt_number,
                IconButton(icons.ADD, on_click=plus_click),
            ],
            alignment="center"
        )
    )

flet.app(target=main)
```

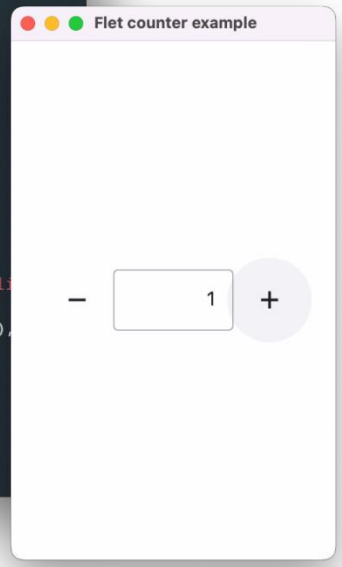
CC BY NC SA

Graphical Interfaces

- Why GUIs
- Building GUIs with FLET
  - Intro to flet library
  - Controllers
- Pattern MVC
  - Motivation and logic of Model-view-controller
  - Code skeleton

Why GUIs

```python
import flet
from flet import IconButton, Page, Row, TextField, icons

def main(page: Page):
    page.title = "Flet counter example"
    page.vertical_alignment = "center"

    txt_number = TextField(value="0", text_align="right",
width=100)

    def minus_click(e):
        txt_number.value = int(txt_number.value) - 1
        page.update()

    def plus_click(e):
        txt_number.value = int(txt_number.value) + 1
        page.update()

    page.add(
        Row(
            [
                IconButton(icons.REMOVE, on_click=minus_cli
                txt_number,
                IconButton(icons.ADD, on_click=plus_click),
            ],
            alignment="center"
        )
    )

flet.app(target=main)
```



- Enhanced user-friendly interaction
- Productivity boost
- Accessibility and inclusivity in software
- Visual representation of data

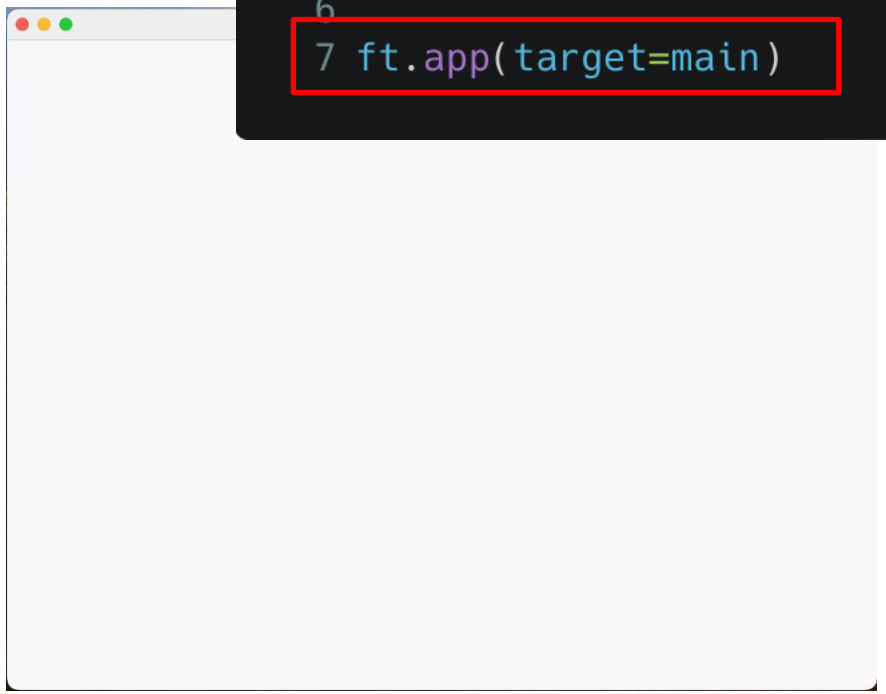
GUIs in Python

- Few alternatives
  - Tk
    - Default graphic library
    - basic set of widgets for building graphical user interfaces.
  - Flutter
    - Proposed by Google as a multi-platform UI software devel kit
  - Flet
    - Library to build flutter apps in python
    - Easily build realtime web, mobile and desktop apps
    - Often prettier, with minimal frontend knowledge ☺
    - https://flet.dev

Let’s start

- Our program will end with a call to flet.app(), where the app starts waiting for new user sessions
- Flet apps are organized in containers, where we put all the graphical elements

```python
import flet as ft

def main(page: ft.Page):
    # add/update controls on Page
    pass

ft.app(target=main)
```



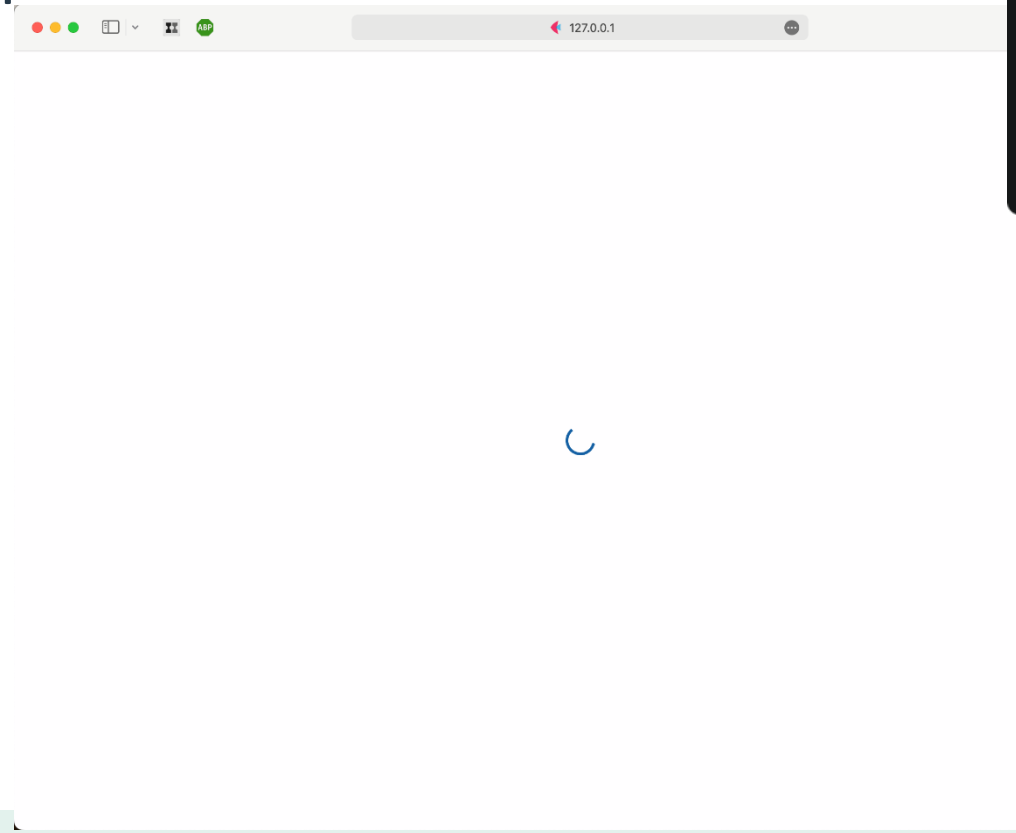
Let’s start

- The top-most container is View (default option is FLET_APP)

```python
import flet as ft

def main(page: ft.Page):
    # add/update controls on Page
    pass

ft.app(target=main, view=ft.AppView.FLET_APP)
```

Let’s start

- But we can easily build a browser app




```python
import flet as ft

def main(page: ft.Page):
    # add/update controls on Page
    pass

ft.app(target=main, view=ft.AppView.WEB_BROWSER)
```

Let’s start

- Within View, we define a Page container, where we will place all the graphic elements.

```python
import flet as ft

def main(page: ft.Page):
    # add/update controls on Page
    pass

ft.app(target=main)
```

GUI elements

- GUI elements are called Controls and are simply python classes which need to be instantiated in a Page (root Control)

- For example, to add some text we can use ft.Text()

GUI elements

- GUI elements are called Controls and are simply python classes which need to be instantiated in a Page (root Control)

- For example, to add some text we can use ft.Text()

```python
import flet as ft

def main(page: ft.Page):
    mytext = ft.Text(value="Ciao amici di TdP2024 :-)", color="Blue")


ft.app(target=main)
```

GUI elements

- GUI elements are called Controls and are simply python classes which need to be instantiated in a Page (root Control)

- For example, to add some text we can use ft.Text()

```python
import flet as ft

def main(page: ft.Page):
    mytext = ft.Text(value="Ciao amici di TdP2024 :-)", color="Blue")
    page.controls.append(mytext)
    page.update()


ft.app(target=main)
```

GUI elements

- GUI elements are called Controls and are simply python classes which need to be instantiated in a Page (root Control)

- Shortcut page.add() to directly add the control and update the page.

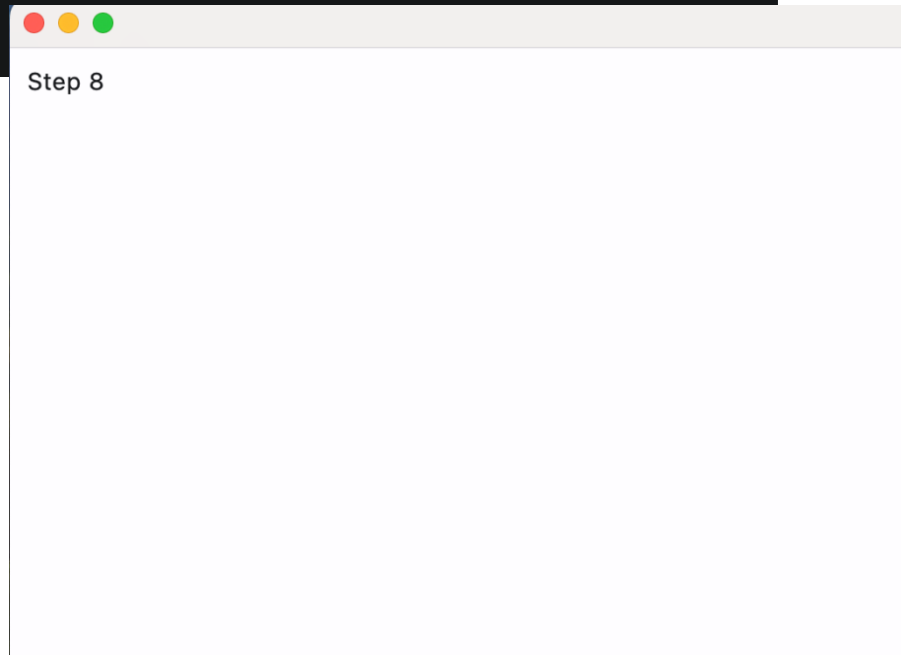
```python
import flet as ft

def main(page: ft.Page):
    mytext = ft.Text(value="Ciao amici di TdP2024 :-)", color="Blue")
    page.controls.append(mytext)
    page.update()

    page.add(ft.Text(value="Quest'anno in Python"))


ft.app(target=main)
```

GUI elements

- Elements in Page are updated only when page.update() is called

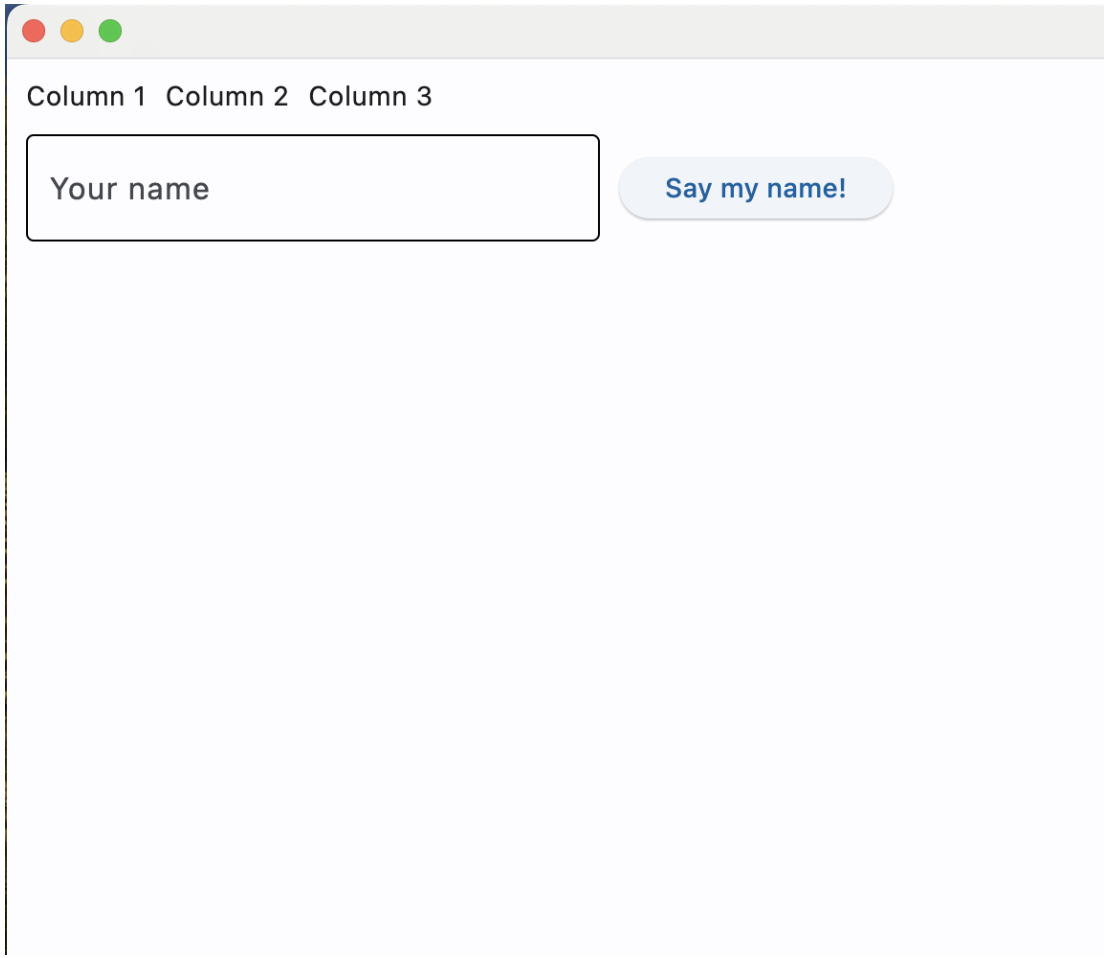
- Note that the content of Controls can be updated anytime, and not only when added to the page, by simply changing the value field

```python
import flet as ft
import time

def main(page: ft.Page):
    t = ft.Text()
    page.add(t)  # it's a shortcut for page.controls.append(t) and then page.update()

    for i in range(10):
        t.value = f"Step {i}"
        page.update()
        time.sleep(1)

ft.app(target=main)
```



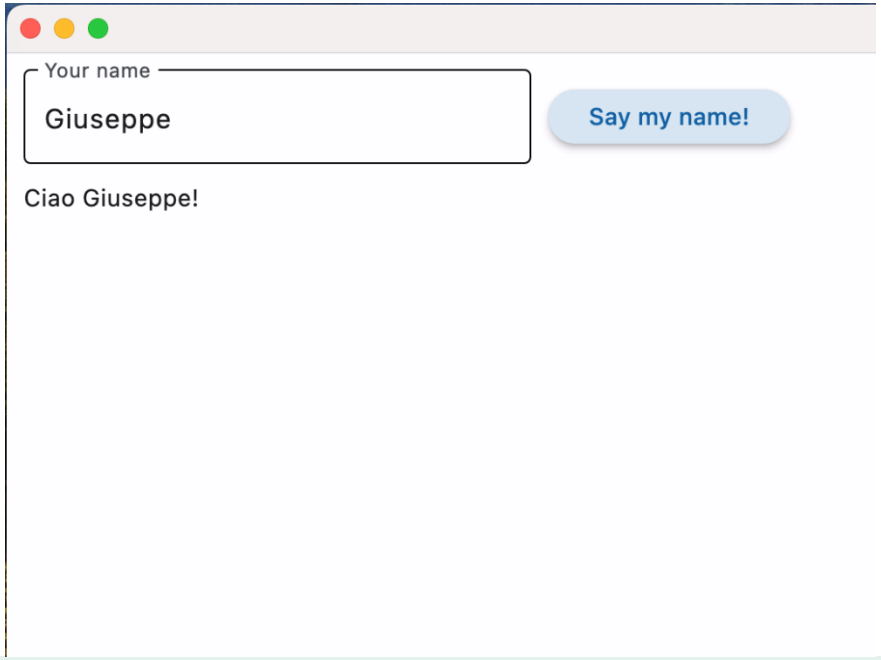
Giving some structure

```python
import flet as ft

def main(page: ft.Page):
    page.add(
        ft.Row(controls=[
            ft.Text("Column 1"),
            ft.Text("Column 2"),
            ft.Text("Column 3")
        ])
    )

    page.add(
        ft.Row(controls=[
            ft.TextField(label="Your name"),
            ft.ElevatedButton(text="Say my name!")
        ])
    )

ft.app(target=main)
```



Event handlers

- Some control items can trigger events:
  - e.g. buttons when clicked




```python
import flet as ft

def main(page: ft.Page):

    def handle_button(e):
        page.add(ft.Text("Ciao " + name.value + "!"))

    name = ft.TextField(label="Your name")
    page.add(
        ft.Row(controls=[
            name,
            ft.ElevatedButton(text="Say my name!", on_click=handle_button)
        ])
    )

ft.app(target=main)
```

Many options

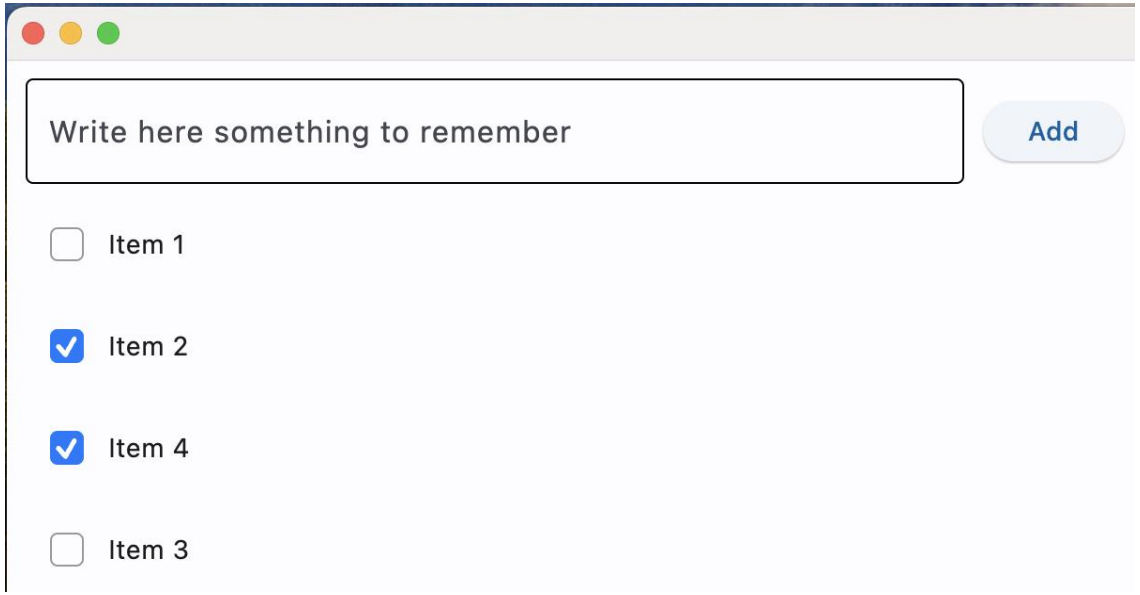
- https://flet.dev/docs/controls

Example: ToDo list

- Let’s try to get something like this

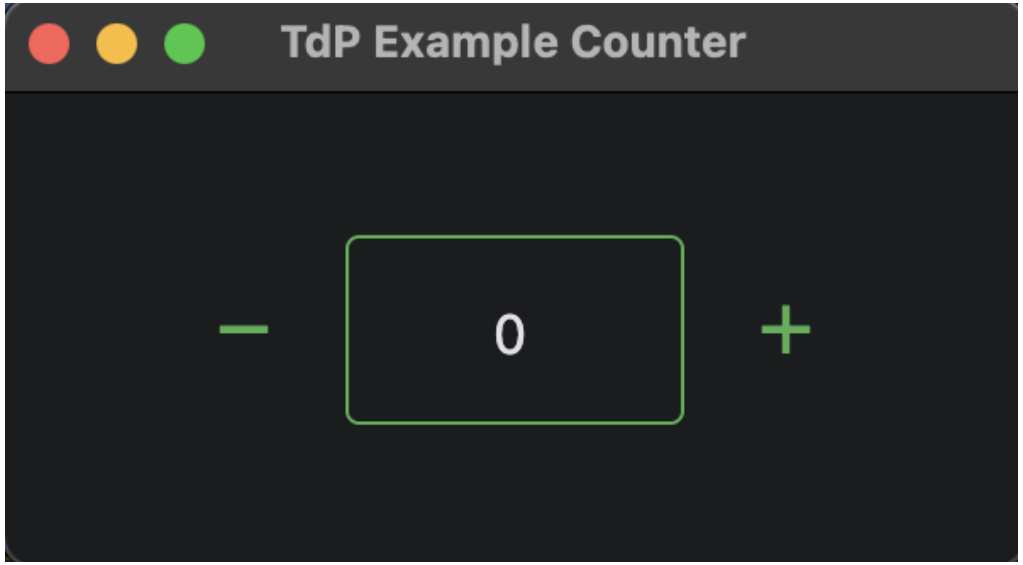
Example: ToDo list

```python
import flet as ft
def main(page):

    def handleAdd(e):
        page.add(ft.CupertinoCheckbox
                 (label=txtBox.value, value=False))
        txtBox.value = ""
        txtBox.update()

    txtBox = ft.TextField(label="Write here something to remember", width=500)
    button = ft.ElevatedButton(text="Add",on_click=handleAdd)
    page.add(ft.Row([txtBox, button]))

ft.app(target=main)
```



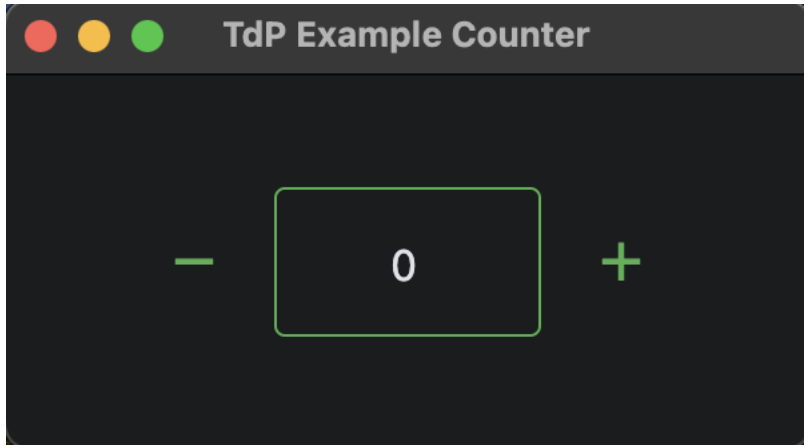
Example: Counter

- Let’s build a counter



Example: Counter

```python
import flet as ft

def main(page: ft.Page):
    page.title = "TdP Example Counter"
    page.vertical_alignment=ft.MainAxisAlignment.CENTER

    def handleRemove(e):
        val.value = val.value-1
        val.update()
    def handleAdd(e):
        val.value = val.value+1
        val.update()

    val = ft.TextField(width=100,border_color="green",value=0,text_align="center")
    page.add(ft.Row([
        ft.IconButton(icon=ft.icons.REMOVE,icon_color="green",on_click=handleRemove),
        val,
        ft.IconButton(icon=ft.icons.ADD,icon_color="green",on_click=handleAdd)
    ],
    alignment=ft.MainAxisAlignment.CENTER
    ))
    page.update()

ft.app(target=main)
```



PATTERN MVC

Application complexity and MVC

- Interactive, graphical applications exhibit complex interaction patterns
- Flow of control is in the hand of the user
- Actions are mainly asynchronous

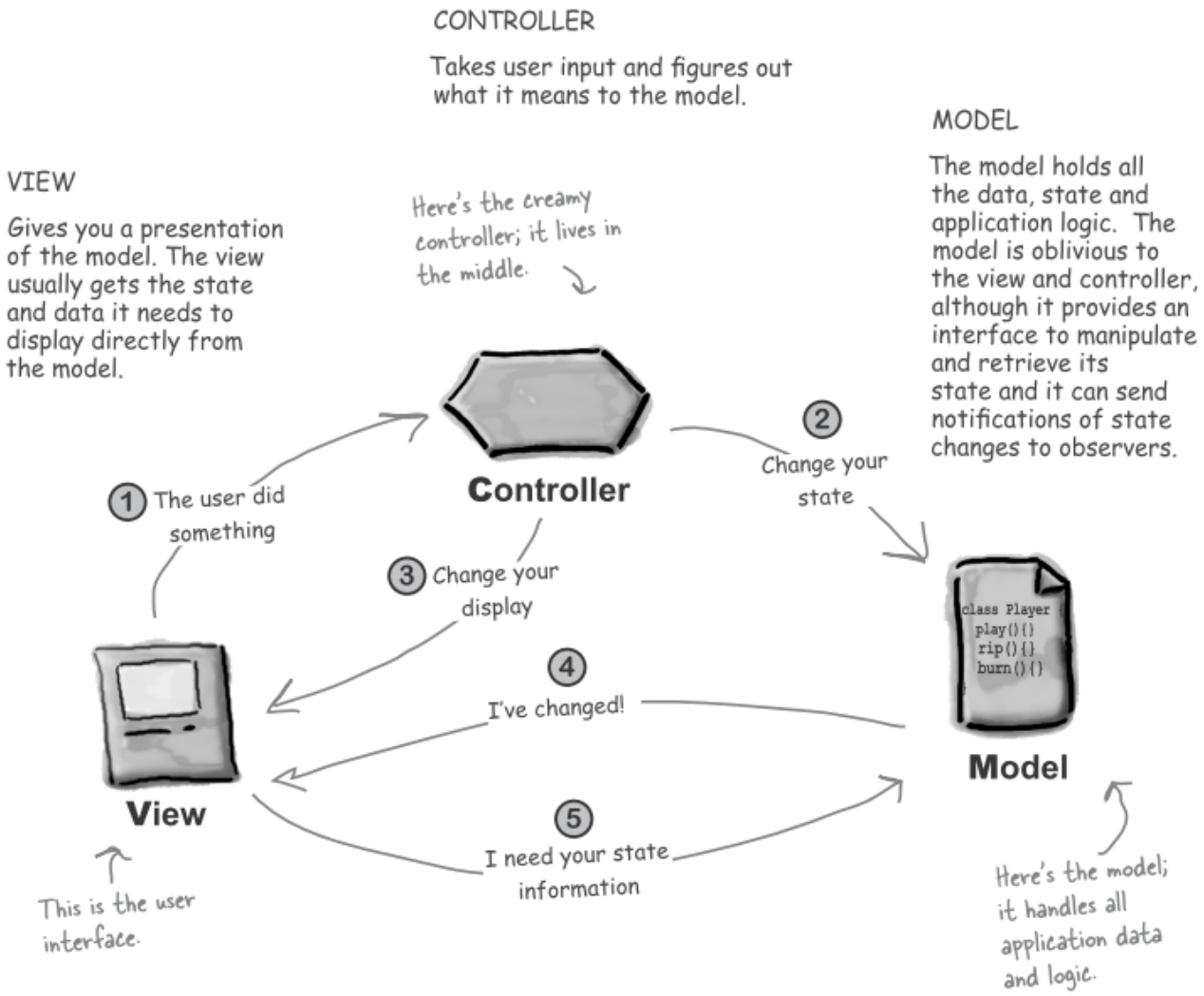
- How to organize the program?
- Where to store data?
- How to decouple application logic from interface details?
- How to keep in sync the inner data with the visibile interface?

Design Patterns

Design Patterns

- How to build systems with good OO design qualities
  - Reusable, extensible, maintainable
- Patterns: Proven solutions to recurrent problems
  - Design problems
  - Programming problems
- Adopt and combine the OO constructs
  - Interface, inheritance, abstract classes, information hiding, polymorphism, objects, statics, …
- Help dealing with changes in software
  - Some part of a system is free to vary, independently from the rest

MVC pattern defined



Normal life-cycle of interaction

1. **You're the user — you interact with the view.**
   The view is your window to the model. When you do something to the view (like click the Play button) then the view tells the controller what you did. It's the controller's job to handle that.

2. **The controller asks the model to change its state.**
   The controller takes your actions and interprets them. If you click on a button, it's the controller's job to figure out what that means and how the model should be manipulated based on that action.

3. **The controller may also ask the view to change.**
   When the controller receives an action from the view, it may need to tell the view to change as a result. For example, the controller could enable or disable certain buttons or menu items in the interface.

4. **The model notifies the view when its state has changed.**
   When something changes in the model, based either on some action you took (like clicking a button) or some other internal change (like the next song in the playlist has started), the model notifies the view that its state has changed.

5. **The view asks the model for state.**
   The view gets the state it displays directly from the model. For instance, when the model notifies the view that a new song has started playing, the view requests the song name from the model and displays it. The view might also ask the model for state as the result of the controller requesting some change in the view.

Mapping concepts to Python

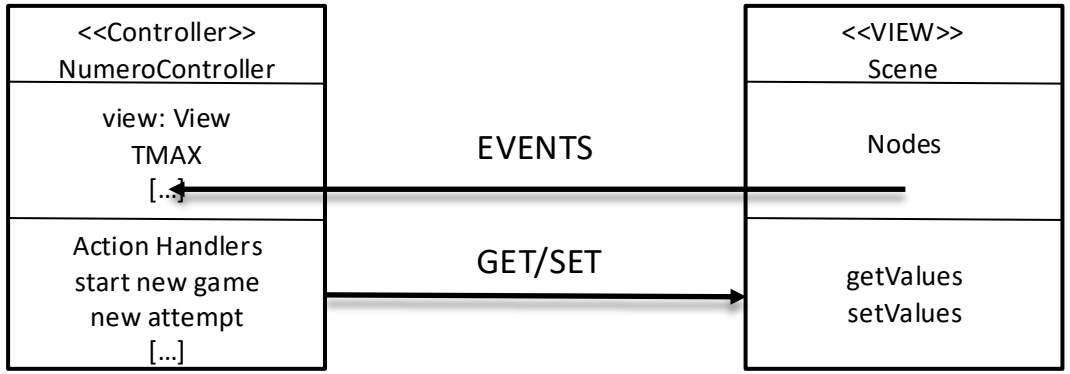
- View: presenting the UI
  - View class, which instantiates controllers defined by FLET
  - interacts with the controller
- Controller: reacting to user actions
  - Set of event handlers
  - Local variable to handle the interface status
- Model: handling the data
  - Class(es) including data
  - Persistent data in Data Bases

Exercise

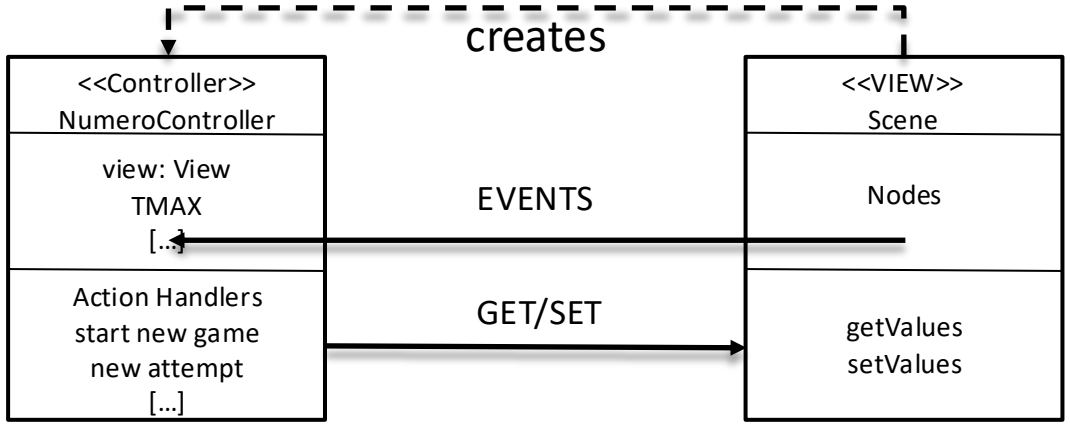
- Update the «IndovinaNumero» by using the MVC pattern

- Where do you declare the data class?
- Which class should have access to which?
- Who creates what objects?

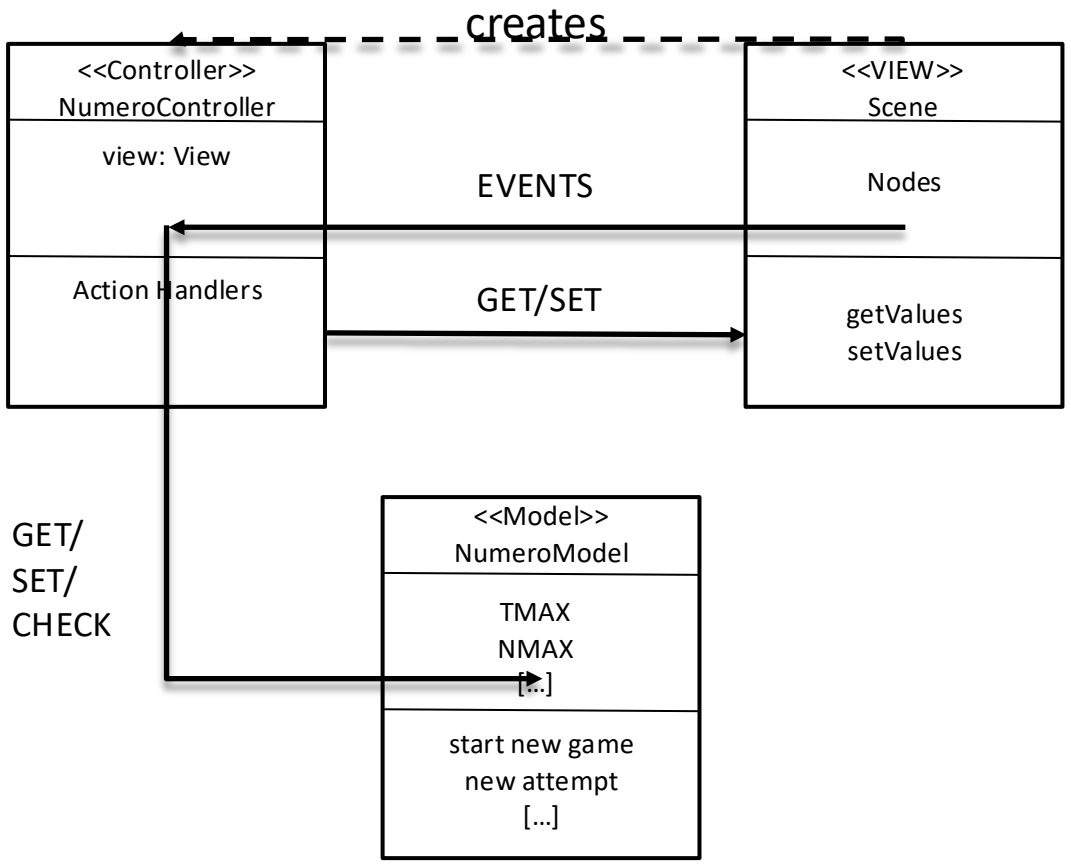
A possible solution



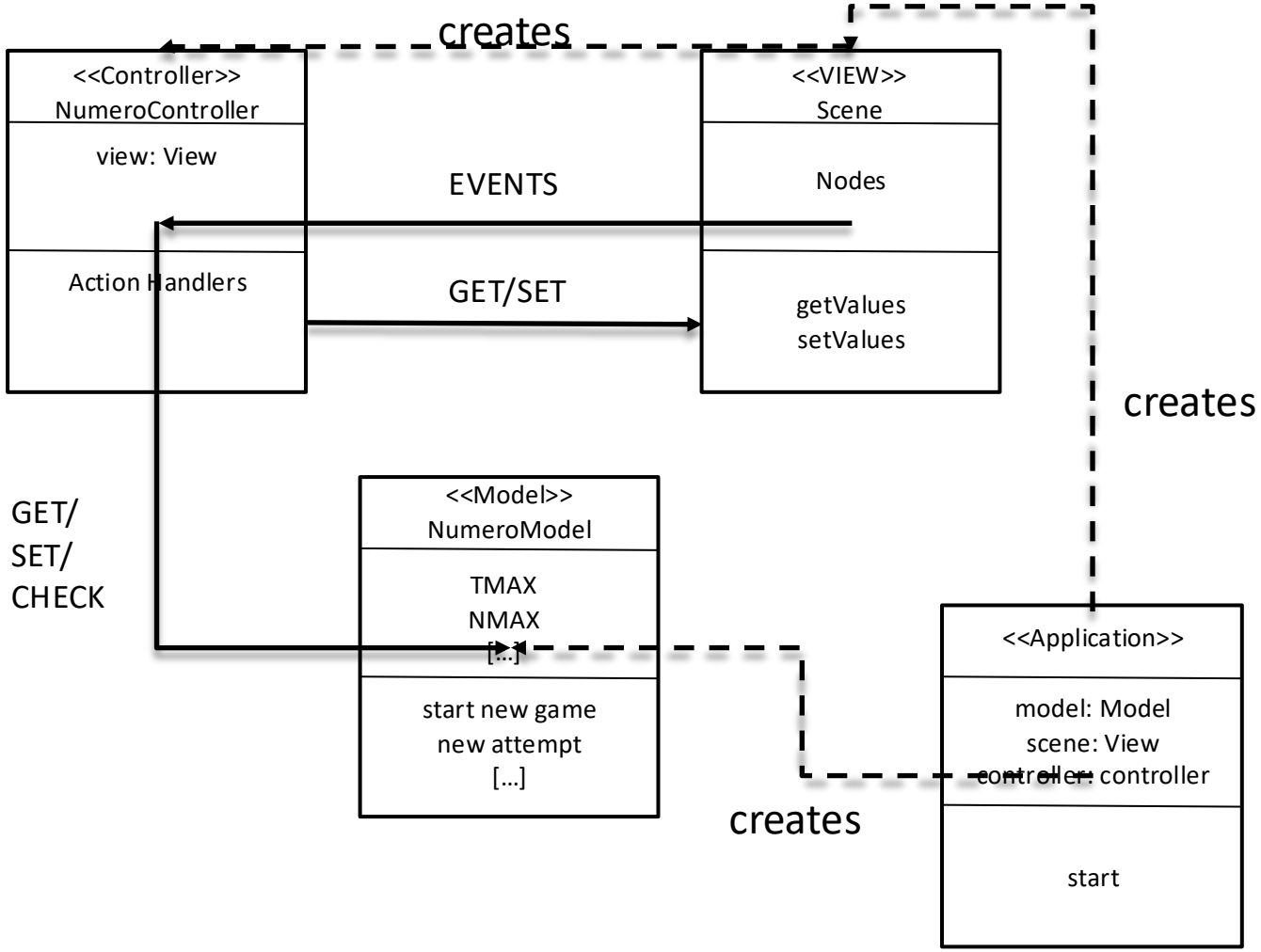
A possible solution



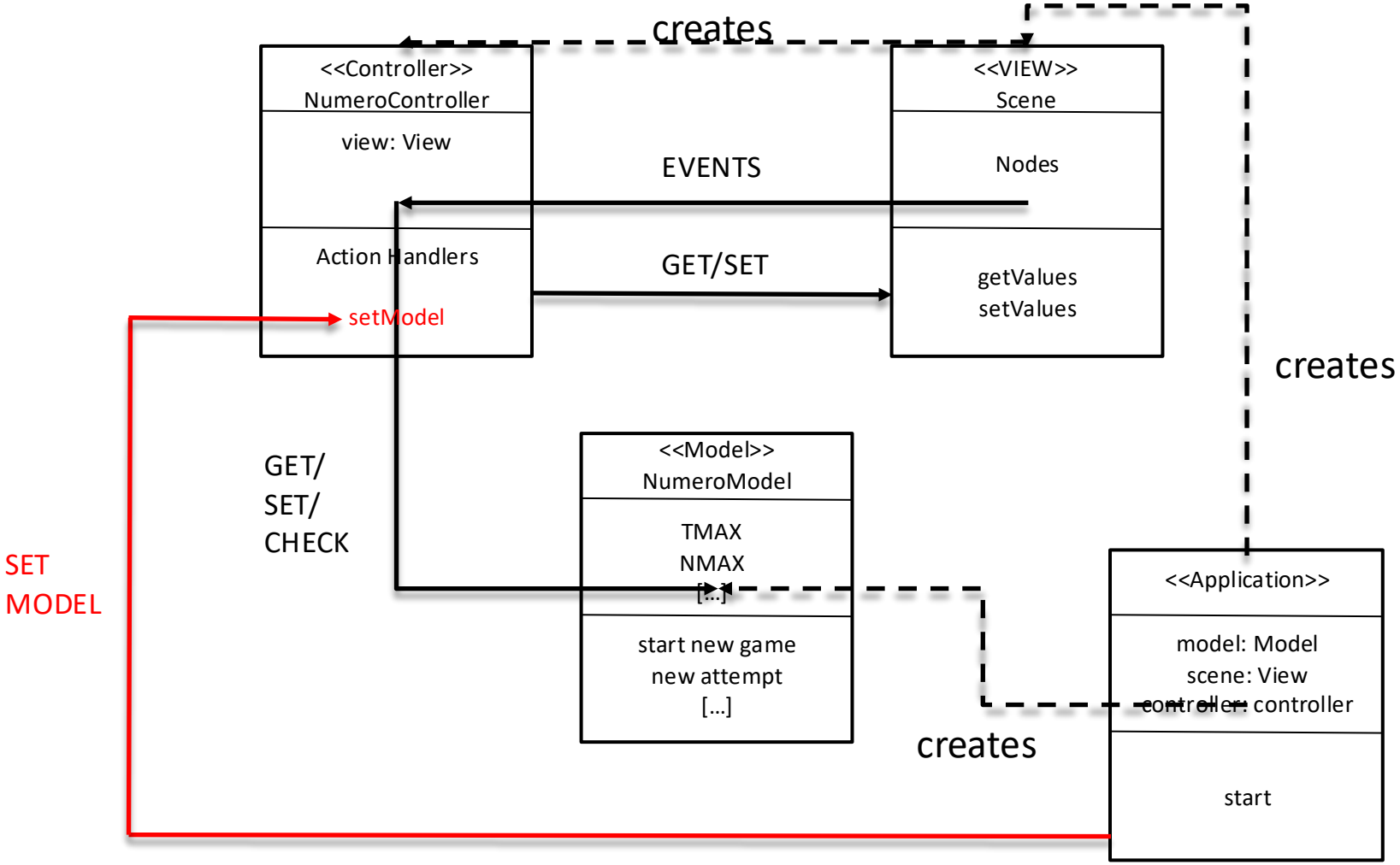
A possible solution



A possible solution

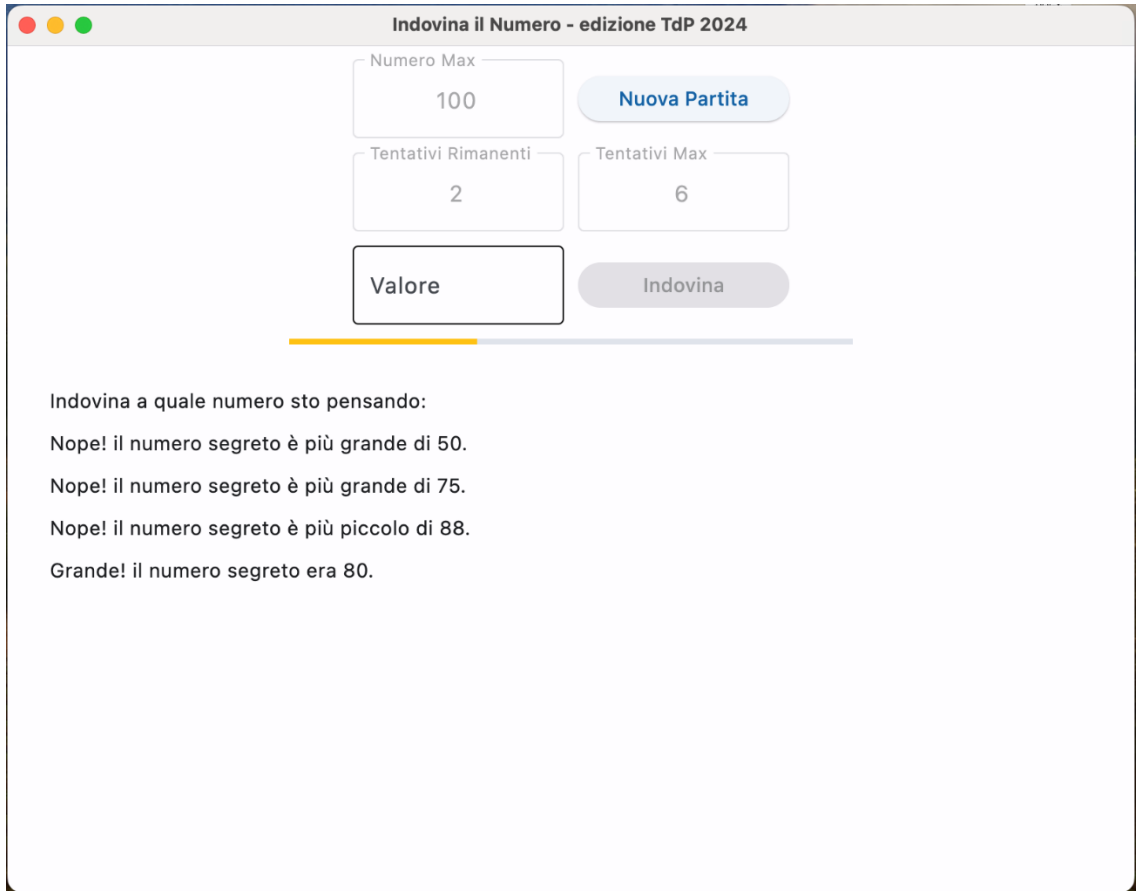


A possible solution



Example: Libretto

Example: Indovina il Numero



License

- These slides are distributed under a Creative Commons license "**Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)**"
- **You are free to:**
  - **Share** — copy and redistribute the material in any medium or format
  - **Adapt** — remix, transform, and build upon the material
  - The licensor cannot revoke these freedoms as long as you follow the license terms.
- **Under the following terms:**
  - **Attribution** — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
  - **NonCommercial** — You may not use the material for commercial purposes.
  - **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.
  - **No additional restrictions** — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.
- https://creativecommons.org/licenses/by-nc-sa/4.0/